TP 1: Génération de la Suite de Fibonacci en Python

Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- Comprendre le principe de la suite de Fibonacci
- Implémenter plusieurs approches algorithmiques
- Comparer la complexité temporelle et spatiale
- Analyser les performances des différentes méthodes
- Comprendre la notion d'optimisation algorithmique

Rappel : Suite de Fibonacci

La suite de Fibonacci est définie comme suit :

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$, pour $n \geq 2$

Exemple :
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Travail demandé

Vous devez implémenter la génération du n-ième terme de la suite de Fibonacci selon **quatre méthodes différentes**.

Pour chaque méthode :

1. Écrire une fonction Python `fibonacci(n)`
2. Tester la fonction pour différentes valeurs de n
3. Mesurer le temps d'exécution
4. Analyser la complexité

Méthode 1 : Approche Itérative

Objectif

Implémenter la suite de Fibonacci en utilisant une boucle (`for` ou `while`).

Contraintes :

- Ne pas utiliser de récursion
- Utiliser seulement deux variables pour stocker les valeurs précédentes
- Complexité attendue : **O(n)**

Questions :

- Pourquoi cette méthode est-elle efficace ?
- Quelle est sa complexité spatiale ?

Méthode 2 : Approche Récursive

Objectif

Implémenter directement la définition mathématique de la suite.

Contraintes :

- Utiliser la récursion pure
- Ne pas utiliser de stockage intermédiaire

Questions :

- Quelle est la complexité temporelle ?
- Pourquoi cette méthode devient-elle très lente pour n > 35 ?
- Combien d'appels récursifs sont effectués ?

Méthode 3 : Programmation Dynamique (Mémoïsation)

Objectif

Optimiser la méthode récursive en mémorisant les résultats déjà calculés.

Contraintes :

- Utiliser un dictionnaire ou `functools.lru_cache`
- Comparer avec la méthode récursive simple

Questions :

- Quelle est la nouvelle complexité ?
- Pourquoi cette méthode est-elle beaucoup plus rapide ?
- Quelle est la différence entre mémoïsation et tabulation ?

Méthode 4 : Formule de Binet

Objectif

Implémenter la formule mathématique fermée :

$$F(n) = \frac{\varphi^n - \psi^n}{\sqrt{5}}$$

avec :

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

$$\psi = \frac{1 - \sqrt{5}}{2}$$

Contraintes :

- Utiliser le module `math`
- Comparer la précision pour les grandes valeurs de n

Questions :

- Pourquoi cette méthode peut-elle donner des erreurs d'arrondi ?
- À partir de quelle valeur observe-t-on une imprécision ?
- Quelle est sa complexité ?

Partie Analyse Comparative

Compléter le tableau suivant :

Méthode	Complexité Temps	Complexité Mémoire	Avantages	Inconvénients
Itérative				
Récursive				
Mémoïsation				
Binet				

Bonus (optionnel)

1. Tracer un graphique du temps d'exécution en fonction de n
2. Implémenter une version avec matrices (puissance rapide)
3. Comparer avec `functools.lru_cache`
4. Tester avec n = 1000

Livrables attendus

- Fichier Python `.py`
- Rapport PDF contenant :
  - Explications
  - Analyse de complexité
  - Captures d'écran des résultats
  - Conclusion comparative

Conclusion attendue

Les étudiants devront conclure :

- Quelle méthode est la plus rapide ?
- Quelle méthode est la plus élégante ?
- Quelle méthode est la plus adaptée en production ?

TP 2: Système de Gestion de Portefeuille d'Investissement en Python (POO)

Contexte

Une société de gestion financière souhaite développer une application permettant de :

- Gérer différents types d'actifs financiers
- Calculer la valeur d'un portefeuille
- Simuler des rendements
- Générer des rapports d'investissement

Vous devez concevoir ce système en utilisant les principes de la **Programmation Orientée Objet (POO)** en Python.

Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- Appliquer les concepts fondamentaux de la POO :
  - Encapsulation
  - Héritage
  - Polymorphisme
  - Abstraction
- Modéliser un problème réel en classes
- Concevoir un système extensible
- Implémenter des cas d'usage métier
- Structurer un projet Python proprement

Concepts POO à mettre en œuvre

1 Encapsulation

- Attributs privés
- Méthodes d'accès (getters / setters si nécessaires)

2 Héritage

- Classe mère `Actif`
- Classes filles :

- Action
- Obligation
- Crypto

3 Polymorphisme

- Méthode commune `calculer_rendement()`
- Chaque type d'actif implémente sa propre logique

4 Abstraction

- Utilisation de classes abstraites (`abc.ABC`)

Travail demandé

Étape 1 : Modélisation des Actifs

Créer une classe abstraite :

```
class Actif(ABC):
    def __init__(self, nom, valeur):
        pass

    @abstractmethod
    def calculer_rendement(self):
        pass
```

Implémenter ensuite :

Classe Action

- Attributs : nom, prix d'achat, prix actuel, dividende, quantité
- Calcul du rendement basé sur variation + dividende

Classe Obligation

- Attributs : nom, valeur nominale, taux d'intérêt, durée
- Calcul du rendement basé sur intérêts

Classe Crypto

- Attributs : nom, prix d'achat, prix actuel, quantité
- Rendement basé uniquement sur volatilité

Étape 2 : Classe Portefeuille

Créer une classe `Portefeuille` permettant :

- Ajouter un actif
- Supprimer un actif
- Calculer la valeur totale
- Calculer le rendement global
- Afficher un résumé du portefeuille

Cas d’usage à implémenter

Cas d’usage 1 : Création d’un portefeuille

L’utilisateur peut créer un portefeuille vide.

Cas d’usage 2 : Ajout d’actifs

L’utilisateur peut ajouter plusieurs types d’actifs.

Cas d’usage 3 : Calcul du rendement global

Le système calcule automatiquement le rendement total du portefeuille.

Cas d’usage 4 : Simulation

Simuler une variation de marché (+5%, -3%, etc.) et recalculer la valeur.

Cas d’usage 5 : Rapport synthétique

Afficher :

- Valeur totale investie
- Valeur actuelle
- Rendement par actif
- Rendement global

Analyse demandée

Les étudiants doivent expliquer :

1. Pourquoi utiliser une classe abstraite ?
2. Où observe-t-on le polymorphisme ?
3. En quoi l'encapsulation protège les données ?
4. Comment rendre le système extensible (ajout futur : ETF, Fonds, etc.) ?

Bonus (optionnel)

- Implémenter une sauvegarde en fichier JSON
- Ajouter une gestion des exceptions
- Implémenter un système de logging
- Ajouter une interface CLI simple
- Appliquer un Design Pattern (ex : Factory pour créer les actifs)

Livrables attendus

1 Code source

- Projet structuré :

```
/portfolio_project
    actif.py
    action.py
    obligation.py
    crypto.py
    portefeuille.py
    main.py
```

2 Rapport PDF (3–5 pages)

Contenant :

- Diagramme UML des classes
- Explication des concepts POO utilisés
- Description des cas d'usage
- Captures d'écran d'exécution
- Analyse critique (forces / limites)

3 Démonstration

Présentation orale de 5–10 minutes :

- Architecture choisie
- Difficultés rencontrées
- Améliorations possibles

Critères d'évaluation

Critère	Points
Respect des concepts POO	/5
Qualité du code	/5
Fonctionnalité correcte	/5
Structure du projet	/3
Rapport et analyse	/2

Total : /20

Résultat attendu

Un système orienté objet propre, extensible et réaliste, simulant un véritable outil de gestion de portefeuille financier.

TP 3: Analyse Exploratoire de Données (EDA) avec Python

Contexte

Une entreprise de e-commerce souhaite analyser ses données de ventes afin de :

- Comprendre le comportement des clients
- Identifier les produits les plus performants
- Détecter des tendances de vente
- Aider à la prise de décision stratégique

Vous êtes chargé(e) de réaliser une **analyse exploratoire des données (EDA)** à partir d'un fichier CSV fourni.

Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- Manipuler des données avec **pandas**
- Nettoyer un jeu de données réel
- Réaliser des statistiques descriptives
- Créer des visualisations pertinentes
- Interpréter les résultats
- Structurer une analyse reproductible

Données fournies

Un fichier `sales_data.csv` contenant par exemple :

Date Client_ID Produit Catégorie Quantité Prix_Unitaire Ville

Outils à utiliser

- Python 3.x
- `pandas`
- `matplotlib` ou `seaborn`
- `numpy`

Travail demandé

Partie 1 : Chargement et Exploration Initiale

1. Charger le fichier CSV dans un DataFrame.
2. Afficher :
    - Les 5 premières lignes
    - Les types de données
    - Les statistiques descriptives
3. Identifier :
    - Les valeurs manquantes
    - Les doublons
    - Les incohérences éventuelles

Questions :

- Combien de lignes et de colonnes contient le dataset ?
- Quelles variables sont numériques ? catégorielles ?

Partie 2 : Nettoyage des Données

1. Traiter les valeurs manquantes :
    - Suppression ou imputation
2. Supprimer les doublons
3. Créer une nouvelle colonne :
    - `Chiffre_Affaires = Quantité × Prix_Unitaire`

Question :

- Quel est l'impact du nettoyage sur le nombre total d'observations ?

Partie 3 : Analyse Descriptive

1. Calculer :
    - Chiffre d'affaires total
    - Vente moyenne par transaction
    - Produit le plus vendu
    - Catégorie la plus rentable
2. Analyser :
    - Répartition des ventes par ville
    - Évolution des ventes dans le temps (mensuelle)

Questions :

- Quelle ville génère le plus de revenus ?
- Existe-t-il une saisonnalité ?

Partie 4 : Visualisation des Données

Créer au minimum :

- Un histogramme des ventes
- Une courbe d'évolution temporelle
- Un diagramme circulaire des catégories
- Un boxplot pour détecter les valeurs aberrantes

Question :

- Quels insights peut-on tirer des visualisations ?

Partie 5 : Analyse Avancée (Optionnelle)

1. Calculer la corrélation entre variables numériques.
2. Identifier les clients les plus fidèles.
3. Segmenter les ventes par tranche de prix.
4. Implémenter une analyse RFM simplifiée.

Analyse attendue

Les étudiants doivent :

- Interpréter les résultats
- Justifier les choix de nettoyage
- Expliquer les tendances observées
- Proposer des recommandations stratégiques

Livrables attendus

1 Code Python

- Script `.py` ou Notebook Jupyter
- Code structuré et commenté

2 Rapport PDF (4–6 pages)

Contenant :

- Description du dataset
- Méthodologie
- Résultats principaux
- Graphiques intégrés
- Interprétation
- Recommandations business

3 Présentation orale (5–10 min)

- Synthèse des insights clés

Critères d'évaluation

Critère	Points
Nettoyage correct	/4
Analyse pertinente	/5
Visualisations	/4
Interprétation	/4
Qualité du code	/3

Total : /20

Résultat attendu

Une analyse claire, structurée et interprétable permettant d'aider l'entreprise à :

- Optimiser ses ventes
- Identifier ses segments clés
- Améliorer sa stratégie commerciale

sales_data.csv

```
Date,Client_ID,Produit,Catégorie,Quantité,Prix_Unitaire,Ville
2024-01-05,C001,Ordinateur Portable,Électronique,1,1200,Paris
2024-01-07,C002,Souris,Électronique,2,25,Lyon
2024-01-10,C003,Chaise Bureau,Mobilier,1,180,Marseille
2024-01-12,C004,Smartphone,Électronique,1,900,Paris
2024-01-15,C005,Livre Python,Livres,3,35,Toulouse
2024-01-20,C006,Bureau,Mobilier,1,350,Nice
2024-01-22,C007,Écouteurs,Électronique,2,60,Bordeaux
2024-01-25,C008,Tablette,Électronique,1,450,Paris
2024-02-01,C001,Livre Data Science,Livres,2,40,Lyon
2024-02-03,C009,Clavier,Électronique,1,70,Paris
2024-02-05,C010,Chaise Bureau,Mobilier,2,180,Lille
2024-02-08,C011,Smartphone,Électronique,1,850,Marseille
2024-02-12,C012,Lampe,Mobilier,3,45,Paris
2024-02-15,C013,Ordinateur Portable,Électronique,1,1150,Bordeaux
2024-02-18,C014,Livre Python,Livres,1,35,Nantes
2024-02-20,C015,Tablette,Électronique,2,420,Toulouse
2024-03-02,C002,Souris,Électronique,3,25,Lyon
2024-03-05,C016,Bureau,Mobilier,1,370,Paris
2024-03-08,C017,Smartphone,Électronique,2,880,Nice
2024-03-12,C018,Livre Data Science,Livres,1,40,Paris
2024-03-15,C019,Écouteurs,Électronique,4,55,Marseille
2024-03-18,C020,Ordinateur Portable,Électronique,1,1250,Paris
2024-03-20,C021,Lampe,Mobilier,2,50,Lyon
2024-03-25,C022,Clavier,Électronique,2,75,Bordeaux
2024-04-01,C003,Tablette,Électronique,1,460,Paris
2024-04-03,C004,Livre Python,Livres,2,35,Nice
2024-04-05,C005,Smartphone,Électronique,1,920,Paris
2024-04-08,C006,Bureau,Mobilier,1,360,Marseille
2024-04-10,C007,Ordinateur Portable,Électronique,1,1180,Lyon
2024-04-15,C008,Chaise Bureau,Mobilier,1,200,Paris
2024-04-18,C009,Souris,Électronique,5,20,Toulouse
2024-04-22,C010,Livre Data Science,Livres,3,45,Bordeaux
2024-05-02,C011,Tablette,Électronique,2,430,Paris
2024-05-05,C012,Lampe,Mobilier,4,40,Nice
2024-05-08,C013,Clavier,Électronique,2,70,Marseille
2024-05-12,C014,Smartphone,Électronique,1,910,Lille
2024-05-15,C015,Ordinateur Portable,Électronique,1,1300,Paris
2024-05-18,C016,Livre Python,Livres,1,30,Nantes
2024-05-22,C017,Bureau,Mobilier,1,380,Paris
2024-06-01,C018,Écouteurs,Électronique,3,65,Lyon
2024-06-04,C019,Tablette,Électronique,1,470,Paris
2024-06-07,C020,Livre Data Science,Livres,2,42,Marseille
2024-06-10,C021,Chaise Bureau,Mobilier,2,190,Paris
2024-06-15,C022,Smartphone,Électronique,1,950,Nice
2024-06-18,C001,Clavier,Électronique,1,75,Lyon
2024-06-22,C002,Lampe,Mobilier,3,48,Bordeaux
2024-07-01,C003,Ordinateur Portable,Électronique,1,1220,Paris
2024-07-04,C004,Souris,Électronique,2,22,Toulouse
2024-07-08,C005,Livre Python,Livres,4,33,Paris
2024-07-12,C006,Smartphone,Électronique,1,890,Marseille
2024-07-15,C007,Bureau,Mobilier,1,400,Paris
2024-07-18,C008,Tablette,Électronique,2,440,Lyon
2024-07-22,C009,Écouteurs,Électronique,3,58,Paris
2024-08-01,C010,Livre Data Science,Livres,1,38,Nice
2024-08-05,C011,Chaise Bureau,Mobilier,1,210,Paris
2024-08-10,C012,Smartphone,Électronique,2,930,Lyon
2024-08-15,C013,Ordinateur Portable,Électronique,1,1280,Paris
```

```
2024-08-18,C014,Souris,Électronique,4,24,Marseille
2024-08-22,C015,Lampe,Mobilier,2,55,Bordeaux
2024-09-01,C016,Tablette,Électronique,1,480,Paris
2024-09-05,C017,Livre Python,Livres,2,36,Nantes
2024-09-10,C018,Clavier,Électronique,3,72,Paris
2024-09-15,C019,Smartphone,Électronique,1,970,Lille
2024-09-18,C020,Bureau,Mobilier,1,420,Paris
2024-09-22,C021,Écouteurs,Électronique,2,62,Lyon
2024-10-01,C022,Ordinateur Portable,Électronique,1,1350,Paris
2024-10-05,C001,Livre Data Science,Livres,3,44,Marseille
2024-10-10,C002,Smartphone,Électronique,1,990,Paris
2024-10-15,C003,Chaise Bureau,Mobilier,1,220,Nice
2024-10-20,C004,Souris,Électronique,3,28,Paris
2024-10-25,C005,Tablette,Électronique,2,460,Lyon
```